

```
anil@anilkumar:~$ cat /proc/ioprocs | grep -v '^$'
0000-001f : dma1
0020-0021 : pic1
0040-0043 : timer0
0050-0053 : timer1
0060-0060 : keyboard
0064-0064 : keyboard
0070-0071 : rtc0
0080-008f : dma page reg
00a0-00a1 : pic2
00c0-00c7 : dma2
00c8-00cf : i804
0078-007a : parport0
03c0-03cf : vesafb
03f0-03ff : serial
0400-047f : 0000:00:1f.0
0500-053f : 0000:00:1f.0
0680-06ff : pnp 00:06
0720-077a : parport0
0c80-0c8f : PCI conf1
1000-1fff : PCI Bus 0000:06
2000-2fff : PCI Bus 0000:02
3000-301f : 0000:00:1f.3
3020-303f : 0000:00:1d.2
3040-305f : 0000:00:1d.1
3060-307f : 0000:00:1d.0
3080-309f : 0000:00:1a.1
```

Figure 1: x86-specific I/O ports

```
val = inw(SERIAL_PORT_BASE + UART_DLL /* 0 */);
outw(val, SERIAL_PORT_BASE + UART_DLL /* 0 */);
```

Blinking an LED

To get a real experience of low-level hardware access and Linux device drivers, the best way would be to play with the Linux device driver kit (LDDK) mentioned above. However, just for a feel of low-level hardware access, a blinking light emitting diode (LED) may be tried, as follows:

- Connect a light-emitting diode (LED) with a 330 ohm resistor in series across Pin 3 (Tx) and Pin 5 (Gnd) of the DB9 connector of your PC.
- Pull up and down the transmit (Tx) line with a 500 ms delay, by loading and unloading the *blink_led* driver, using *insmod blink_led.ko* and *rmmod blink_led*, respectively.

Driver file *blink_led.ko* can be created from its source file *blink_led.c* by running *make* with the usual driver Makefile. Given below is the complete *blink_led.c*:

```
#include <linux/module.h>
#include <linux/version.h>
#include <linux/types.h>
#include <linux/delay.h>
#include <asm/io.h>

#include <linux/serial_reg.h>

#define SERIAL_PORT_BASE 0x3FB

int __init init_module()
{
    int i;
    u8 data;

    data = inb(SERIAL_PORT_BASE + UART_LCR);
    for (i = 0; i < 5; i++)
```

Figure 2: Registers of UART PC16550D

```
{
    /* Pulling the Tx line low */
    data |= UART_LCR_SBC;
    outb(data, SERIAL_PORT_BASE + UART_LCR);
    msleep(500);
    /* Defaulting the Tx line high */
    data &= ~UART_LCR_SBC;
    outb(data, SERIAL_PORT_BASE + UART_LCR);
    msleep(500);
}
return 0;
}

void __exit cleanup_module()
{
}

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Anil Kumar Pugalia <email@sarika-pugs.com>");
MODULE_DESCRIPTION("Blinking LED Hack");
```

Looking ahead

You might have wondered why Shweta is missing from this article? She bunked all the classes! Watch out for the next article to find out why.

By: Anil Kumar Pugalia

The author is a freelance trainer in Linux internals, Linux device drivers, embedded Linux and related topics. Prior to this, he was at Intel and Nvidia. He has been working with Linux since 1994. A gold medalist from the Indian Institute of Science, Linux and knowledge sharing are two of his many passions. Creating and playing with open source hardware is one of his hobbies. Learn more about him at <http://profession.sarika-pugs.com>, including information about his open source hardware freak-outs and his various Linux related training sessions and workshops. He can be reached at email@sarika-pugs.com.